Sistemi Operativi – a.a. 2025/2026

prova di laboratorio

– 29 settembre 2026 –

Creare un programma `crc-records-checker-2.c` in linguaggio C che accetti invocazioni sulla riga di comando del tipo:

```
crc-records-checker-2 <M-verifiers> <file-bin-1> ... <file-bin-N>
```

Il programma prende in input N≥1 file binari contenenti sequenze di record dotati di un checksum di integrità e dovrà contare i record integri e quelli corrotti di ciascun file, verificando inoltre che il numero di record integri effettivamente rilevato coincida con quello dichiarato all'interno di ogni file.

Ogni file binario ha la seguente struttura:

- un header di 4 byte contenente un intero senza segno little-endian che dichiara il numero di record integri presenti nel file (usare il tipo C `uint32_t` in `stdint.h`);
- una sequenza di record binari di 16 byte ciascuno: i primi 15 byte rappresentano i dati, l'ultimo byte rappresenta il checksum del record; i byte sono da intendersi senza segno e quindi da trattare con il tipo C `uint8_t` (o `unsigned char`).

Un record si dice **integro** se il byte di checksum coincide con lo **XOR bit a bit** (operatore ^ del C) dei 15 byte di dati; in caso contrario il record si dice **corrotto**.

Sono disponibili dei file di esempio: `records-1.bin`, `records-2-bad.bin`, `records-3.bin`.

In particolare al suo avvio il programma creerà N + M thread ausiliari:

- N thread **lettori** che si occuperanno, rispettivamente, di leggere uno dei file indicati sulla riga di comando; il file dovrà essere letto un record alla volta tramite le ordinarie chiamate di I/O su file (`open`/`read` oppure `fopen`/`fread`), senza caricarlo interamente in memoria; ogni lettore depositerà preliminarmente nel vettore condiviso il numero di record integri dichiarato nell'header e, per ogni record letto, inserirà un elemento nella coda candidati; la lettura proseguirà fino alla fine del file;
- M thread **verificatori** che si occuperanno, in modo concorrente, di estrarre, uno alla volta, i record dalla coda candidati e di ricalcolarne il checksum: se questo coincide con quello dichiarato nel record, sarà incrementato il contatore degli integri del file di provenienza; in caso contrario sarà incrementato il contatore dei corrotti.

Le strutture dati condivise saranno:

- una **coda candidati** di elementi con capienza massima di 10 elementi che conterrà i record letti dai file da verificare;
- un vettore di N terne <dichiarati, integri, corrotti> con i conteggi relativi a ciascun file in input;
- mutex e **variabili condizione**: il loro numero, comunque minimale, e le modalità d'uso sono da determinare da parte dello studente;
- eventuali flag/contatori di fine-lavoro.

Il singolo elemento della coda conterrà le seguenti informazioni:

- i 16 byte del record (15 di dati + 1 di checksum);
- l'identificativo del file di provenienza;
- il numero progressivo del record all'interno del file.

Il thread principale/padre, dopo aver creato tutti i thread ausiliari, ne attenderà la terminazione; al termine, per ogni file, confronterà il numero di record integri rilevato con quello dichiarato nell'header riportando l'esito della verifica automatica (superata o fallita) e il riepilogo finale.

L'uso di mutex e variabili condizione è obbligatorio: soluzioni che non li utilizzino saranno considerate non corrette. Tutti i thread dovranno terminare spontaneamente alla fine dei lavori e non si dovranno usare strutture dati con visibilità globale. È necessario rispettare fedelmente

la struttura dell'output riportato nell'esempio a seguire. Codici sorgente con errori che bloccano la compilazione e pregiudicano la generazione del codice binario con compilatori standard (gcc o clang) non saranno valutati.

Tempo: 2 ore

La struttura dell'output atteso è la seguente:

```
$ ./crc-records-checker-2 2 records-1.bin records-2-bad.bin records-3.bin
[MAIN] creazione di 3 thread lettori e 2 thread verificatori
[READER-1] file 'records-1.bin': 60 record dichiarati integri
[READER-1] record candidato n.1 (offset 4)
[READER-1] record candidato n.2 (offset 20)
[READER-1] record candidato n.3 (offset 36)
[READER-2] file 'records-2-bad.bin': 47 record dichiarati integri
[READER-2] record candidato n.1 (offset 4)
[READER-3] file 'records-3.bin': 271 record dichiarati integri
[READER-3] record candidato n.1 (offset 4)
[VERIF-1] record n.1 del file 'records-1.bin': checksum dichiarato 63, calcolato 63 -> integro
[VERIF-1] record n.2 del file 'records-1.bin': checksum dichiarato 239, calcolato 48 -> corrotto
[VERIF-2] record n.1 del file 'records-3.bin': checksum dichiarato 231, calcolato 231 -> integro
[VERIF-1] record n.1 del file 'records-2-bad.bin': checksum dichiarato 134, calcolato 134 -> integro
[READER-3] record candidato n.2 (offset 20)
[VERIF-2] record n.3 del file 'records-1.bin': checksum dichiarato 53, calcolato 53 -> integro
...
[READER-2] terminazione con 55 record letti
[READER-1] terminazione con 70 record letti
[READER-3] terminazione con 300 record letti
[VERIF-1] terminazione con 264 record verificati
[VERIF-2] terminazione con 161 record verificati
[MAIN] file 'records-1.bin': 60 integri e 10 corrotti su 70 record -> verifica automatica superata
(dichiarati 60)
[MAIN] file 'records-2-bad.bin': 46 integri e 9 corrotti su 55 record -> verifica automatica fallita
(dichiarati 47, rilevati 46)
[MAIN] file 'records-3.bin': 271 integri e 29 corrotti su 300 record -> verifica automatica superata
(dichiarati 271)
[MAIN] verifiche superate: 2/3
[MAIN] terminazione
```